ISIM
Institut Supérieur d'Informatique de Médenine

Année Universitaire
2025-2026

Cahier des Charges

Projet TaskFlow - Gestionnaire de Tâches Collaboratif

Classe : MP1ILC **Enseignant :** M. JARRAY Mondher **Framework :** Laravel 11 **Version :** 1.0

1. Présentation du Projet

Objectif Général

TaskFlow est une application web de gestion de tâches collaborative développée avec Laravel 11. Elle permet aux utilisateurs de créer, organiser et suivre leurs tâches personnelles et en équipe, avec un système de catégories, de priorités et de notifications.

1.1 Contexte Pédagogique

Ce projet est développé progressivement à travers les 10 TPs du cours de Développement Web Avancé. Chaque TP apporte de nouvelles fonctionnalités :

TP 1-2 — **Fondations**
Installation Laravel, structure du projet, routes de base, premiers contrôleurs

TP 3-4 — **Données**
Base de données, migrations, modèles Eloquent, relations entre entités

TP 5-6 — **Interface**
Vues Blade, layouts, formulaires, validation des données

TP 7-8 — **Sécurité & API**
Authentification Breeze, autorisation (Gates/Policies), API REST avec Sanctum

TP 9-10 — **Avancé**
Events, Notifications, Queues, Tests PHPUnit, Déploiement

2. Spécifications Fonctionnelles

2.1 Gestion des Utilisateurs

Authentification (Laravel Breeze)

- Inscription avec email et mot de passe
- Connexion / Déconnexion
- Réinitialisation de mot de passe par email
- Vérification d'email
- Profil utilisateur modifiable (nom, email, avatar)

> **User Story :** En tant qu'utilisateur, je veux pouvoir créer un compte et me connecter pour accéder à mes tâches personnelles.

2.2 Gestion des Tâches (CRUD Complet)

Fonctionnalités des Tâches

- **Création :** Titre, description, date d'échéance, priorité, catégorie
- **Liste :** Affichage avec filtres (statut, catégorie, priorité), tri, pagination
- **Modification :** Édition de tous les champs
- **Suppression :** Soft delete avec possibilité de restauration
- **Changement de statut :** pending → in_progress → completed

> **User Story :** En tant qu'utilisateur, je veux créer des tâches avec une priorité et une date limite pour organiser mon travail efficacement.

2.3 Gestion des Catégories

Fonctionnalités des Catégories

- Création de catégories personnalisées (nom, couleur, icône)
- Attribution d'une catégorie à chaque tâche
- Filtrage des tâches par catégorie
- Statistiques par catégorie

> **User Story :** En tant qu'utilisateur, je veux organiser mes tâches par catégories (Travail, Personnel, Études) pour mieux les visualiser.

2.4 Tableau de Bord

Dashboard

- Vue d'ensemble des tâches (total, en cours, terminées)
- Tâches urgentes (deadline proche)
- Statistiques visuelles (graphiques)
- Activité récente

2.5 Système de Notifications

Notifications

- Rappel avant date d'échéance (email + base de données)
- Notification lors de l'assignation d'une tâche
- Centre de notifications dans l'interface
- Marquage lu/non lu

> **User Story :** En tant qu'utilisateur, je veux recevoir un rappel par email lorsqu'une tâche approche de sa deadline.

2.6 API REST

Endpoints API (Sanctum)

Méthode	Endpoint	Description
POST	`/api/login`	Authentification, retourne token
POST	`/api/logout`	Déconnexion, révoque token
GET	`/api/tasks`	Liste des tâches de l'utilisateur
POST	`/api/tasks`	Créer une nouvelle tâche
GET	`/api/tasks/{id}`	Détails d'une tâche
PUT	`/api/tasks/{id}`	Modifier une tâche
DELETE	`/api/tasks/{id}`	Supprimer une tâche
GET	`/api/categories`	Liste des catégories

3. Spécifications Techniques

3.1 Stack Technologique

Backend
Laravel 11
PHP 8.2+

Frontend
Blade + Tailwind CSS
Alpine.js

Base de données
MySQL 8.0 / SQLite

Authentification
Laravel Breeze
Sanctum (API)

Build Tools
Vite
NPM

Tests
PHPUnit
Pest (optionnel)

3.2 Modèle de Données

users
- id (PK)
- name
- email
- password
- avatar
- is_admin
- email_verified_at
- timestamps

tasks
- id (PK)
- title
- description
- status
- priority
- due_date
- completed_at
- user_id (FK)
- category_id (FK)
- timestamps
- deleted_at

categories
- id (PK)
- name
- color
- icon
- user_id (FK)
- timestamps

Relations

- **users** 1:N **tasks** (un utilisateur possède plusieurs tâches)
- **users** 1:N **categories** (un utilisateur possède plusieurs catégories)
- **categories** 1:N **tasks** (une catégorie contient plusieurs tâches)

3.3 Structure des Fichiers

```
taskflow/
  app/
    Http/Controllers/
      TaskController.php
      CategoryController.php
      DashboardController.php
```

```
      Api/
        TaskApiController.php
    Models/
      User.php, Task.php, Category.php
    Policies/
      TaskPolicy.php
    Events/
      TaskAssigned.php
    Listeners/
      SendTaskAssignedNotification.php
    Notifications/
      TaskDueReminder.php
  database/migrations/
  resources/views/
    layouts/, tasks/, categories/, dashboard/
  routes/
    web.php, api.php
  tests/
    Feature/, Unit/
```

4. Règles de Gestion

RG1 - Propriété des tâches

Un utilisateur ne peut voir, modifier ou supprimer que ses propres tâches (sauf admin).

RG2 - Statuts des tâches

Une tâche passe par les statuts : **pending** → **in_progress** → **completed**. Le retour en arrière est possible.

RG3 - Priorités

Les priorités sont : **low** (1), **medium** (2), **high** (3), **urgent** (4). Par défaut : medium.

RG4 - Date d'échéance

La date d'échéance doit être supérieure ou égale à la date du jour lors de la création.

RG5 - Catégories

Chaque utilisateur gère ses propres catégories. Une catégorie ne peut être supprimée que si aucune tâche ne lui est associée.

RG6 - Notifications

Un rappel est envoyé 24h avant la date d'échéance pour les tâches non terminées.

RG7 - Soft Delete

Les tâches supprimées sont archivées (soft delete) et peuvent être restaurées pendant 30 jours.

5. Livrables Attendus

Livrables du Projet

- 📁 Code source complet du projet Laravel
- 📁 Base de données avec seeders et factories
- 📁 Documentation API (collection Postman ou similaire)
- 📁 Tests unitaires et fonctionnels (couverture min. 60%)
- 📁 README avec instructions d'installation
- 📁 Démonstration fonctionnelle de l'application

Critères d'Évaluation

- Fonctionnalités complètes et opérationnelles
- Qualité du code (structure, conventions, commentaires)
- Interface utilisateur (UX/UI)
- Tests et documentation
- Bonus : Fonctionnalités supplémentaires